The Compositionality–Atomicity Tradeoff

Technical Report

Ahmad Al-Zaro

https://github.com/ahmadalzaro1/tokenization-shapes-embeddings

Abstract

We present a controlled mechanistic study of how tokenization design shapes learned representations in language models. Using Arabic diacritical marks as a natural test case, we compare three encoding strategies on the same corpus: standard BPE on diacritized text (d1, compositional), harakat-stripped BPE (d2, lossy), and a novel atomic encoding mapping each letter+diacritic pair to a unique token (d3, atomic). Across 200+ architecture-search experiments and a tokenizer-fair metric (Bits Per Base Letter), the atomic encoding—despite achieving zero fragmentation and 15% lower fertility—yields higher loss than standard BPE under constrained model and search budgets. Mechanistic analysis of the embedding layer reveals a contributing mechanism: atomic tokens have near-random cosine similarity (0.125) between diacritical variants of the same letter, indicating the embedding layer does not organize diacritical variants into compositional clusters. Iso-data scaling experiments show the disadvantage is data-dependent, shrinking from 25% at 5M base letters to 1.7% at 500M. We term this the *compositionality–atomicity tradeoff*: atomic tokenization eliminates fragmentation but forces the model to learn relationships from scratch rather than inheriting them from shared subword structure.

1 Introduction

أَلَمٌ أَلَمَّ أَلَمْ أُلِمَّ بِدَائِهِ
إِنْ أَنَّ آنٌّ آنَ آنُ أَوَانِهِ

"A pain befell—was I not told of its ailment? / If a sufferer moaned, the time of its season has come."

Without diacritics, all four words in the first hemistich reduce to *alm* (الم).

— Anonymous, often misattributed to Al-Mutanabbi

How does the design of a tokenizer shape what a language model learns internally? Tokenization is typically treated as a preprocessing step—a practical engineering choice that determines sequence length and vocabulary size but is assumed to be neutral with respect to the model's learned representations. We show this assumption is wrong.

Arabic diacritical marks (*harakat*) provide a natural controlled experiment. The same Arabic text can be encoded three ways without changing its linguistic content: (1) preserving diacritics as Unicode combining characters, (2) stripping them entirely, or (3) fusing each letter+diacritic pair into a single atomic token. These three strategies produce identical information content but radically different tokenizations, enabling a clean causal test of how tokenizer design propagates into the model's embedding space.

We designed the atomic encoding (d3) to be "perfect": each Arabic letter and its attached diacritic form a single indivisible token, eliminating the fragmentation problems that plague BPE on morphologically rich languages [13]. The result is zero subword splitting of diacritical information and 15% lower fertility than standard diacritized BPE. By every conventional metric, d3 should win.

It does not. Across 200+ architecture-search experiments, 10-seed robust evaluation, and a tokenizer-fair metric we introduce (Bits Per Base Letter, bpbl), standard diacritized BPE (d1) consistently outperforms atomic encoding (d3), which in turn outperforms harakat-stripped BPE (d2). The ordering d1 $<$ d3 $<$ d2 (lower is better) is preserved under fixed-architecture transfer, across multiple seeds, and across data scales from 5M to 500M base letters.

We explain this result mechanistically by analyzing the learned embedding matrices. In d3, diacritical variants of the same base letter—tokens that *should* be related—have near-random cosine similarity (mean 0.125). At the embedding level, the model treats *ba* (b+fathah) and *bu* (b+dammah) as nearly unrelated tokens. In contrast, d1 learns structured embeddings where harakah tokens cluster together (cosine 0.224), well-separated from base letter tokens. This points to a contributing mechanism: atomic tokenization removes the compositional structure that BPE preserves through shared subwords.

Iso-data scaling experiments reveal the tradeoff is not permanent. The d1 advantage shrinks from 25% at 5M base letters to 1.7% at 500M, suggesting d3 can eventually learn the compositional relationships from co-occurrence statistics—it just needs far more data. We term this the *compositionality–atomicity tradeoff*: a general principle that applies wherever tokenizer design choices trade fragmentation reduction against compositional structure.

Contributions.

- Mechanistic evidence that tokenization design shapes embedding structure, demonstrated through controlled embedding analysis across three Arabic encoding strategies.
- bpbl (Bits Per Base Letter): a tokenizer-fair evaluation metric that removes the fertility confound inherent in standard bits-per-byte comparisons.
- Characterization of the compositionality–atomicity tradeoff as data-scale dependent: a 25% gap at 5M letters that narrows to 1.7% at 500M letters.
- Practical guidance for Arabic NLP: do not strip diacritics; do not use atomic encoding at small data scales.

2 Background

Arabic diacritical marks. Arabic script consists of 28 base consonant letters. Short vowels are indicated by *harakat*—diacritical marks placed above or below letters. These include *fathah* (a), *dammah* (u), *kasrah* (i), their tanween variants (*fathatan*, *dammatan*, *kasratan*), *shaddah* (gemination), *sukun* (vowelless), and *superscript alef*. In Unicode, harakat are encoded as combining characters (U+064B–U+0652, U+0670) that attach to the preceding base letter.

Modern Arabic text is overwhelmingly written without harakat. This creates massive lexical ambiguity: the undiacritized form *ktb* maps to *kataba* ('he wrote'), *kutiba* ('it was written'), *kutub* ('books'), and other forms depending on context. Most Arabic NLP systems strip diacritics during preprocessing [6], treating harakat as noise. Fully vocalized text exists primarily in the Quran, classical literature, and educational materials.

BPE tokenization. Byte Pair Encoding [16] is the dominant subword tokenization algorithm. BPE iteratively merges the most frequent character pairs, producing a vocabulary of subword units. For Arabic with diacritics, BPE must handle combining characters, which can lead to fragmentation: a single diacritized letter may be split across multiple tokens. Prior work has shown that BPE performs poorly on morphologically rich languages [13, 9], motivating alternative tokenization strategies.

Arabic tokenization in practice. Current Arabic language models universally strip diacritics during preprocessing. Jais [15] removes "rare diacritic marks" via CAMeL Tools normalization and trains a custom BPE tokenizer achieving 1.05 tokens/word on undiacritized Arabic. AraBERT [1], AraGPT2 [2], and CAMeLBERT [8] all strip harakat as a default preprocessing step. ALLaM [5] does not discuss diacritics handling. This is a rational engineering choice: web-scale Arabic text is overwhelmingly undiacritized, so preserving harakat would waste vocabulary slots on tokens rarely seen in training data. The sole exception is Fanar [11], which explicitly reserves vocabulary entries for diacritics with the intent of enabling vocalized generation during fine-tuning. However, no Arabic LLM paper has systematically compared diacritized vs. undiacritized training on language modeling quality. We address this gap: given fully vocalized text, how should it be encoded?

Tokenization beyond compression. Recent work challenges the assumption that better tokenizer compression yields better models. [14] train 64 models and show that compression ratio alone does not predict downstream quality; pre-tokenization choices and vocabulary initialization matter more. MorphBPE [4] constrains BPE merges to respect morpheme boundaries and shows consistent cross-entropy reduction for Arabic, demonstrating that linguistically informed tokenization can outperform standard BPE even at matched compression. These findings provide context for our result that d3's lower fertility does not translate to lower loss.

Mechanistic interpretability. Mechanistic interpretability aims to understand neural networks by analyzing their internal representations [7, 10]. We adopt this perspective to study how tokenization propagates into the embedding layer—the first learned representation in a language model. Rather than comparing tokenizers only on downstream performance, we open the model and examine what structure the embeddings actually learn.

3 Experimental Setup

3.1 Corpus and Conditions

We use the Tashkeela corpus [17], a collection of fully vocalized classical Arabic text (1.5M examples). From this corpus, we produce three parallel datasets differing only in how the same text is encoded:

d1 (Compositional): Fully diacritized text in standard Unicode. Each harakah is a combining character following its base letter. BPE may merge letters with their diacritics or keep them separate, depending on corpus statistics.

d2 (Lossy): Harakat stripped. This represents the standard Arabic NLP preprocessing pipeline. Information about short vowels, gemination, and case endings is permanently lost.

d3 (Atomic): Each base letter + attached harakah pair is mapped to a unique codepoint in the Unicode Private Use Area (PUA, U+E000–U+EFFF). This produces 327 unique atomic tokens covering all 33 base letters and their observed diacritical variants. BPE sees each diacritized letter as an indivisible unit—there is zero fragmentation of diacritical information.

This design isolates tokenizer design from corpus content: all three conditions encode identical linguistic information (except d2, which is intentionally lossy).

Ambiguity statistics. To quantify the cost of stripping diacritics, we compute homograph collision statistics at corpus scale. Of 2,256,984 unique undiacritized forms, 608,287 (26.95%) map to multiple diacritized forms, with an average collision rate of 2.07. In a 128-token context window, 96.77% of tokens belong to ambiguous forms. Stripping harakat is not a neutral preprocessing step.

Condition	val_bpb ↓	Depth	Fert.
d1 (comp.)	**0.660**	4	2.519
d3 (atomic)	0.890	2	2.193
d2 (lossy)	1.020	4	1.467

Table 1: Architecture search results across 200+ experiments. Each condition finds its own optimal architecture. d1 achieves the lowest (best) validation loss despite higher fertility than d3.

3.2 Model Architecture

All experiments use a GPT-style decoder-only transformer [12] implemented in MLX [3] on Apple Silicon. Models have ∼5M parameters with vocabulary size 8,192. The architecture uses RoPE positional encoding, grouped-query attention with KV heads, and a sliding-window attention pattern.

3.3 Architecture Search

We run 70+ architecture-search experiments per condition (200+ total), varying depth (2–6 layers), aspect ratio (width/depth), head dimension (64–128), and attention window pattern. Each run trains for a fixed 5-minute budget on a single Apple Silicon device. This search allows each encoding to find its own optimal architecture rather than being constrained to a single shared design.

3.4 Bits Per Base Letter (bpbl)

Standard bits-per-byte (bpb) is confounded by fertility differences across tokenizers. d2 has fertility 1.467 tokens/word while d1 has 2.519 and d3 has 2.193 (at vocabulary size 8,192). A model that processes fewer tokens per word appears better in bpb simply because it sees fewer prediction opportunities.

We introduce Bits Per Base Letter (bpbl):

$$\text{bpbl} = \frac{\sum_t \ell_t \cdot \mathbf{1}[b_t > 0]}{\ln 2 \cdot \sum_t \beta_t} \tag{1}$$

where ℓ_t is the cross-entropy loss for token t, b_t is the byte count of token t, β_t is the number of Arabic base letters in token t, and the sum runs over all tokens in the evaluation set. bpbl measures prediction quality per unit of Arabic linguistic content, removing the fertility confound.

3.5 Evaluation Protocol

We evaluate models at two levels of rigor:

1. **Architecture search:** Single seed per configuration, selecting the best bpb per condition.
2. **Robust evaluation:** 10 seeds per condition with the best architecture, reporting median bpbl (robust to outlier seeds).

4 Results

4.1 Architecture Search

Table 1 shows the best validation bpb per condition after architecture search.

The result is surprising: d3, designed for zero fragmentation, loses to d1 by a substantial margin (0.890 vs. 0.660). Furthermore, the conditions prefer different architectures: d1 and d2 favor depth-4 networks, while d3 prefers a shallower depth-2 network with a larger aspect ratio.

Shared Arch	d1	d3	d2	Order
d1 winner	**0.701**	0.838	1.052	d1<d3<d2
d2 winner	**0.681**	0.930	1.037	d1<d3<d2
d3 winner	**1.037**	1.290	1.404	d1<d3<d2

Table 2: Fixed-architecture transfer matrix (val_bpb). The ordering d1<d3<d2 is preserved under all three shared architectures.

	Cond.	Mean	Med.↓	Std
Original	d1	3.332	**2.933**	0.908
	d3	3.673	3.412	0.751
Replication	d1	2.811	**2.724**	0.293
	d3	3.301	2.873	0.926

Table 3: bpbl results across 10 seeds per condition (original and independent replication with fresh seeds). We report median as the robust central tendency due to outlier seeds (see §7). Both runs confirm d1 outperforms d3 on median bpbl.

4.2 Fixed-Architecture Transfer

A potential confound is that the ordering reflects architecture differences rather than encoding differences. We address this with a transfer matrix: each condition's best architecture is used to train all three conditions (Table 2).

The ordering d1 $<$ d3 $<$ d2 survives all three shared architectures, confirming the result is driven by encoding design, not architecture differences.

4.3 Robust bpbl Evaluation

We train 10 seeds per condition using each condition's optimal architecture and evaluate with bpbl. Table 3 reports the results.

The median bpbl confirms the ordering: d1 (2.933) substantially outperforms d3 (3.412), a 14% advantage. The high standard deviations reflect outlier seeds (seed 7 for d1, seed 137 for d3), motivating our use of median over mean.

5 Mechanistic Analysis

Why does d3 lose despite having "perfect" tokenization? We answer this by analyzing the learned embedding matrices of trained models.

5.1 D3 Embedding Analysis

For d3, we extract the token embedding matrix ($W_{te} \in \mathbb{R}^{V \times d}$) from the best-performing seed and compute intra-group cosine similarity for each of the 33 base letters. Each group contains all PUA tokens derived from the same base letter (e.g., all diacritical variants of *bā'*).

The results (Table 4) are striking: the mean intra-group cosine similarity across all 33 base letters is **0.125** ($\sigma = 0.062$). For context, random vectors in the embedding space would have expected cosine similarity near zero; 0.125 is only marginally above random. At the embedding level, the model treats diacritical variants of the same base letter as nearly independent tokens.

Consider the letter *beh*: its 11 diacritical variants (with fathah, dammah, kasrah, shaddah+fathah, etc.) have mean cosine similarity 0.079. The embedding layer has no structural representation of the fact that these tokens all share the base letter *beh*. We note that compositional relationships could in

Base Letter	Variants	Cosine Sim
beh	11	0.079
teh	11	0.095
dal	14	0.099
reh	13	0.097
lam	14	0.080
meem	12	0.057
noon	11	0.125
yeh	15	0.034
Overall (33 letters)	**327**	**0.125**

Table 4: Intra-group cosine similarity in d3 embeddings for selected base letters. The overall mean across all 33 base letters (327 PUA tokens) is 0.125—near the random baseline. At the embedding level, diacritical variants of the same letter lack compositional structure.

principle emerge in later layers through attention; our analysis establishes that the embedding layer does not provide this structure as an inductive bias, unlike d1.

5.2 D1 Embedding Analysis

In d1, harakah are separate combining-character tokens in the vocabulary. We analyze the 9 singleton harakah tokens (fathah, dammah, kasrah, fathatan, dammatan, kasratan, shaddah, sukun, superscript alef).

The mutual cosine similarity among harakah tokens is **0.224**, substantially higher than d3's intra-group similarity (0.125). The cosine similarity between harakah tokens and base letter tokens is only 0.044, showing clear separation. d1 learns a compositional structure: harakah form a coherent cluster in embedding space, separate from base letters.

5.3 The Compositionality–Atomicity Tradeoff

This analysis reveals a likely mechanism behind d3's disadvantage. By fusing each letter+harakah pair into an atomic token, d3 eliminates the shared subword structure through which BPE enables parameter sharing. In d1, seeing the fathah token in any context provides information about fathah in all contexts, because the *same* embedding is reused. In d3, the model must independently learn that *beh+fathah*, *teh+fathah*, *dal+fathah*, etc. all carry similar diacritical information—a relationship that is obvious from d1's shared tokens but invisible in d3's atomic representation.

We call this the *compositionality–atomicity tradeoff*: atomic tokenization eliminates fragmentation (a real benefit) but destroys compositional parameter sharing (a real cost). The net effect depends on how much data is available to learn the compositional relationships from scratch.

6 Scaling Analysis

If d3's disadvantage stems from needing to learn compositional structure from data rather than inheriting it from shared subwords, the gap should shrink with more data. We test this prediction with iso-data scaling experiments.

6.1 Iso-Data Protocol

We train d1 and d3 models at 7 data scales (5M, 15M, 30M, 50M, 100M, 200M, 500M base letters) with 3 seeds per point, using each condition's optimal architecture. "Base letters" count only Arabic consonant letters, providing a tokenizer-agnostic measure of data volume.

Base Letters	d1	d3	Gap
5M	$4.06_{\pm.06}$	$5.09_{\pm.12}$	25.2%
15M	$3.23_{\pm.02}$	$3.85_{\pm.07}$	19.0%
30M	$2.93_{\pm.01}$	$3.15_{\pm.04}$	7.7%
50M	$2.75_{\pm.07}$	$2.91_{\pm.01}$	6.1%
100M	$2.59_{\pm.03}$	$2.72_{\pm.02}$	5.2%
200M	$2.49_{\pm.02}$	$2.56_{\pm.01}$	2.9%
500M	$2.44_{\pm.01}$	$2.48_{\pm.01}$	1.7%

Table 5: Iso-data scaling: mean bpbl ($\pm$ std) across 3 seeds. The gap shrinks monotonically from 25.2% at 5M base letters to 1.7% at 500M. Both conditions use their own optimal architecture. An independent replication confirms all values within $\leq$3% (Appendix D).

Base Letters	d1 opt	d3 opt	d3+d1 arch
5M	4.060	5.085	4.134
50M	2.747	2.914	2.760
200M	2.487	2.558	2.505
500M	2.439	2.481	2.450

Table 6: Architecture confound control. d3 with d1's deeper architecture (depth-4) performs comparably to d1 at large data scales, and substantially better than d3 with its own shallower architecture. The encoding effect diminishes; architecture matters more at scale.

6.2 Scaling Results

Table 5 and Figure 2 confirm the prediction. The d1 advantage shrinks monotonically from 25.2% at 5M base letters to 1.7% at 500M. The gap closes rapidly between 5M and 30M (from 25% to 8%), then continues to narrow more slowly. At 500M base letters, d3 nearly matches d1 (2.481 vs. 2.439 bpbl).

This pattern is consistent with the compositionality–atomicity tradeoff: d1's advantage is a data-efficiency benefit from compositional parameter sharing, not a fundamental representational limitation of d3. Given sufficient data, d3 can learn the compositional relationships from co-occurrence statistics. An independent replication of all 42 scaling runs with fresh random seeds confirms these results within $\leq$3% at every scale (Appendix D).

6.3 Architecture Confound Control

A potential concern is that the scaling comparison conflates encoding effects with architecture effects, since d1 uses a depth-4 model while d3 uses depth-2. We run an additional condition: d3 with d1's architecture (depth-4). Table 6 shows results at selected scales.

At small data scales (5M), d3 with d1's architecture already performs comparably to d1 (4.134 vs. 4.060), far better than d3 with its own architecture (5.085). At 500M, all three conditions converge (2.439, 2.481, 2.450). This suggests that much of d3's measured disadvantage at the "optimal" architecture is actually an architecture effect: the shallower model preferred by d3 in the 5-minute search is suboptimal for larger data budgets (Figure 3).

7 Discussion

Scope of the encoding effect. Our architecture confound control (§6) reveals an important nuance: when d3 uses d1's deeper architecture, the performance gap nearly vanishes at all scales. This means a meaningful portion of the measured disadvantage reflects the interaction between encoding and architecture search—the 5-minute search prefers a shallower model for d3 that becomes suboptimal at larger

data budgets. The encoding effect proper (at matched architecture) is real but smaller than the headline numbers suggest, and is best characterized as a data-efficiency disadvantage under constrained search budgets rather than a fundamental limitation of atomic tokenization.

A hypothesis for further testing. The compositionality–atomicity tradeoff may generalize beyond Arabic. Any morphologically rich language (Turkish, Finnish, Hebrew, Hindi) faces analogous design decisions: should morphological features be encoded as separate tokens (compositional) or fused into atomic units? Our results suggest that compositional encoding is more data-efficient because it enables parameter sharing in the embedding layer, but atomic encoding may be viable at sufficient data scale. Confirming this hypothesis requires experiments on additional languages, larger models, and downstream tasks.

Implications for Arabic NLP. Current Arabic LLMs strip diacritics because their training data is overwhelmingly undiacritized—a rational choice for web-scale models [15, 1, 8]. Our results speak to a different setting: when vocalized text *is* available—Quranic and classical Arabic, text-to-speech, diacritization models, and educational applications—the recommendation is clear: *do not strip diacritics*. The lossy encoding (d2) is consistently worst across all experiments. Between d1 and d3, d1 is preferable at small to moderate data scales. At very large scales (hundreds of millions of base letters and beyond), the difference becomes negligible. Fanar's decision to reserve vocabulary entries for diacritics [11] is consistent with our findings.

Tokenization as an interpretability tool. Our approach demonstrates that controlled tokenization experiments can serve as a mechanistic interpretability tool. By comparing models trained on the same content with different tokenizations, we can isolate how tokenizer design shapes internal representations—a complementary perspective to post-hoc probing and attention analysis.

Scaling predictions. The monotonically decreasing gap suggests that d3 may eventually match or surpass d1 at larger model and data scales, where the embedding layer's compositional prior becomes less important relative to the model's capacity to learn compositional structure from context. We cannot confirm this prediction within our experimental budget, but the trend is consistent across all measured scales.

Limitations

Model scale. All experiments use $\sim$5M parameter models. The compositionality–atomicity tradeoff may behave differently at larger model scales (hundreds of millions or billions of parameters) where the embedding layer is a smaller fraction of total parameters.

Corpus scope. We use a single corpus (Tashkeela) of fully vocalized classical Arabic. Results may differ for modern standard Arabic, dialectal Arabic, or partially vocalized text.

No downstream evaluation. We evaluate language modeling quality (bpbl) but not downstream task performance (NER, question answering, machine translation). The relationship between bpbl differences and task performance is not established.

Embedding-layer analysis only. Our mechanistic analysis examines cosine similarity in the token embedding matrix. Low cosine similarity among diacritical variants does not preclude compositional structure emerging in deeper attention or MLP layers. The embedding finding is consistent with the performance gap but is not sufficient on its own to prove a full absence of learned compositional structure.

Seed variance. Despite 10-seed evaluation, the high standard deviation in bpbl (Table 3) indicates sensitivity to random initialization, particularly at small data scales.

Gradient step imbalance. In iso-data scaling, we equalize base letters processed, not gradient steps. Because d1 and d3 have different fertilities, they take different numbers of gradient steps for the same data budget. This is by design (equalizing data, not compute) but represents a confound in the strictest interpretation.

AI-assisted development. All experimental code was developed with AI assistance (Claude, Gemini). Research design, decisions, and interpretation are by the authors.

8 Conclusion

We provide mechanistic evidence that tokenization design shapes learned representations in the embedding layer. Through controlled experiments on Arabic diacritical marks, we show that atomic tokenization disrupts compositional structure in embeddings (cosine similarity 0.125, near-random), creating a data-efficiency disadvantage that shrinks from 25% to under 2% with scale. Architecture confound analysis further shows that much of the measured gap reflects the interaction between encoding and architecture search, narrowing the pure encoding effect.

The compositionality–atomicity tradeoff is a hypothesis supported by our evidence: eliminating token fragmentation comes at the cost of compositional parameter sharing, with the net effect depending on data scale, model architecture, and search budget. bpbl provides a fair metric for cross-tokenizer comparison.

For Arabic NLP practitioners: keep your diacritics. For the broader community: tokenization is not a neutral preprocessing step—it shapes what your model's embedding layer learns, though the effect diminishes with scale.

References

[1] Wissam Antoun, Fady Baly, and Hazem Hajj. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools (OSACT)*, 2020. URL https://aclanthology.org/2020.osact-1.2/.

[2] Wissam Antoun, Fady Baly, and Hazem Hajj. AraGPT2: Pre-trained transformer for Arabic language generation. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop (WANLP)*, 2021. URL https://aclanthology.org/2021.wanlp-1.21/.

[3] Apple Machine Learning Research. MLX: An array framework for Apple silicon. https://github.com/ml-explore/mlx, 2023.

[4] Ehsaneddin Asgari, Youssef El Kheir, and Mohammad Ali Sadraei Javaheri. MorphBPE: A morpho-aware tokenizer bridging linguistic complexity for efficient LLM training across morphologies. *arXiv preprint arXiv:2502.00894*, 2025. URL https://arxiv.org/abs/2502.00894.

[5] M. Saiful Bari et al. ALLaM: Large language models for Arabic and English. *arXiv preprint arXiv:2407.15390*, 2024. URL https://arxiv.org/abs/2407.15390.

[6] Kareem Darwish, Nizar Habash, Mourad Abbas, Hend Al-Khalifa, Huseein T. Al-Natsheh, Samhaa R. El-Beltagy, Houda Bouamor, Karim Bouzoubaa, Violetta Cavalli-Sforza, Wassim El-Hajj, Mustafa Jarrar, and Hamdy Mubarak. A panoramic survey of natural language processing in the Arab world. *Communications of the ACM*, 64(4):72–81, 2021. doi: 10.1145/3447735.

[7] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition. *arXiv preprint arXiv:2209.10652*, 2022. URL https://transformer-circuits.pub/2022/toy_model/index.html.

[8] Go Inoue, Bashar Alhafni, Nurpeiis Baimukan, Houda Bouamor, and Nizar Habash. The interplay of variant, size, and task type in Arabic pre-trained language models. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop (WANLP)*, 2021. URL https://aclanthology.org/2021.wanlp-1.10/.

[9] Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raja, Chenglei Si, Wilson Y. Lee, Benoît Sagot, and Samson Tan. Between words and characters: A brief history of open-vocabulary modeling and tokenization in NLP. *arXiv preprint arXiv:2112.10508*, 2021. URL https://arxiv.org/abs/2112.10508.

[10] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads. *arXiv preprint arXiv:2209.11895*, 2022. URL https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html.

[11] QCRI. Fanar: An Arabic-centric multimodal generative AI platform. *arXiv preprint arXiv:2501.13944*, 2025. URL https://arxiv.org/abs/2501.13944.

[12] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[13] Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. How good is your tokenizer? on the monolingual performance of multilingual language models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3118–3135, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.243. URL https://aclanthology.org/2021.acl-long.243/.

[14] Craig W. Schmidt, Varuni Reddy, Haoran Zhang, Alec Alameddine, Omri Uzan, Yuval Pinter, and Chris Tanner. Tokenization is more than compression. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. URL https://aclanthology.org/2024.emnlp-main.40/.

[15] Neha Sengupta, Sunil Kumar Sahu, Bokun Jia, Satheesh Katiber, Hao Li, Faris Wehbe, Nour Zhuang, et al. Jais and jais-chat: Arabic-centric foundation and instruction-tuned open generative large language models. *arXiv preprint arXiv:2308.16149*, 2023. URL https://arxiv.org/abs/2308.16149.

[16] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1162. URL https://aclanthology.org/P16-1162/.

Letter	n	**Cos**	**Letter**	n	**Cos**	**Letter**	n	**Cos**
hamza	6	0.032	seen	11	0.115	meem	12	0.057
above	6	0.112	sheen	11	0.124	noon	11	0.125
alef	5	0.116	sad	11	0.177	heh	10	0.101
beh	11	0.079	dad	10	0.098	waw	11	0.093
marbuta	6	0.265	tah	11	0.096	maksura	8	0.258
teh	11	0.095	zah	10	0.118	yeh	15	0.034
theh	10	0.156	ain	8	0.092			
jeem	10	0.159	ghain	4	0.278		**Overall**	
hah	10	0.040	feh	10	0.078	**33 letters, 327 tokens**		**0.125**
khah	9	0.104	qaf	10	0.120			
dal	14	0.099	kaf	10	0.137			
thal	10	0.131	lam	14	0.080			
reh	13	0.097						
zain	11	0.168						

Table 7: Complete per-letter intra-group cosine similarity in d3 embeddings. n = number of diacritical variants for each base letter. Letters with fewer variants (e.g., ghain: 4, marbuta: 6) tend to have higher cosine similarity due to less distributional diversity.

[17] Taha Zerrouki and Amar Balla. Tashkeela: Novel corpus of Arabic vocalized texts, data for auto-diacritization systems. *Data in Brief*, 11:147–151, 2017. doi: 10.1016/j.dib.2017.01.011.

A Full Per-Letter Cosine Similarity

Table 7 reports the intra-group cosine similarity for all 33 base letters in the d3 embedding analysis.

B Scaling Curves

Figure 4 shows the D3 disadvantage as a percentage gap, shrinking monotonically with data scale.

C Architecture Search Details

The architecture search varied the following hyperparameters:

- **Depth**: 2, 3, 4, 5, 6 layers
- **Aspect ratio**: 8, 12, 16, 24, 26, 32, 48, 64
- **Head dimension**: 64, 96, 128
- **Window pattern**: S (sliding), SS, SSS, SSSL (sliding+global)
- **Batch size**: 2^{13}, 2^{14}, 2^{15} tokens

All configurations use vocabulary size 8,192, sequence length 2,048, RoPE positional encoding, and grouped-query attention. Each run trains for exactly 5 minutes on Apple Silicon with the MLX framework.

	Original		Replication	
BL	**d1**	**d3**	**d1**	**d3**
5M	4.060	5.085	4.177	4.969
15M	3.232	3.846	3.209	3.752
30M	2.928	3.154	2.909	3.143
50M	2.747	2.914	2.737	2.923
100M	2.588	2.723	2.593	2.695
200M	2.487	2.558	2.493	2.562
500M	2.439	2.481	2.428	2.482

Table 8: Iso-data scaling replication. Original and replicated mean bpbl across 3 seeds per condition. Maximum absolute deviation between original and replication is 2.9% (at 5M for d1). The ordering d1<d3 replicates at all 7 scales.

D Reproducibility

All core experiments were independently replicated with fresh random seeds on the same hardware (Apple Silicon, MLX framework). The replication comprised 42 iso-data scaling runs (7 budgets $\times$ 2 conditions $\times$ 3 seeds) and 14 additional bpbl evaluation runs (7 new seeds $\times$ 2 conditions), totaling 9.6 hours of compute.

Iso-data scaling replication. Table 8 compares original and replicated mean bpbl at each data scale. All values deviate by less than 3%, and the ordering d1 $<$ d3 is preserved at every scale.

Robust bpbl replication. The 10-seed robust evaluation (Table 3) was replicated with a fresh set of random seeds. Both runs confirm d1 outperforms d3 on median bpbl: original 2.933 vs. 3.412 (14.0% gap); replication 2.724 vs. 2.873 (5.5% gap). The narrower gap in the replication reflects reduced seed variance with the new seed set, but the ordering is preserved.

Embedding analysis. The embedding similarity metrics are deterministic given a trained model and replicate exactly: d3 intra-group cosine similarity = 0.125, d1 harakah mutual cosine = 0.224.

Summary. All three core claims—d1<d3 at all scales, monotonically decreasing gap, and near-random d3 embeddings—replicate independently.

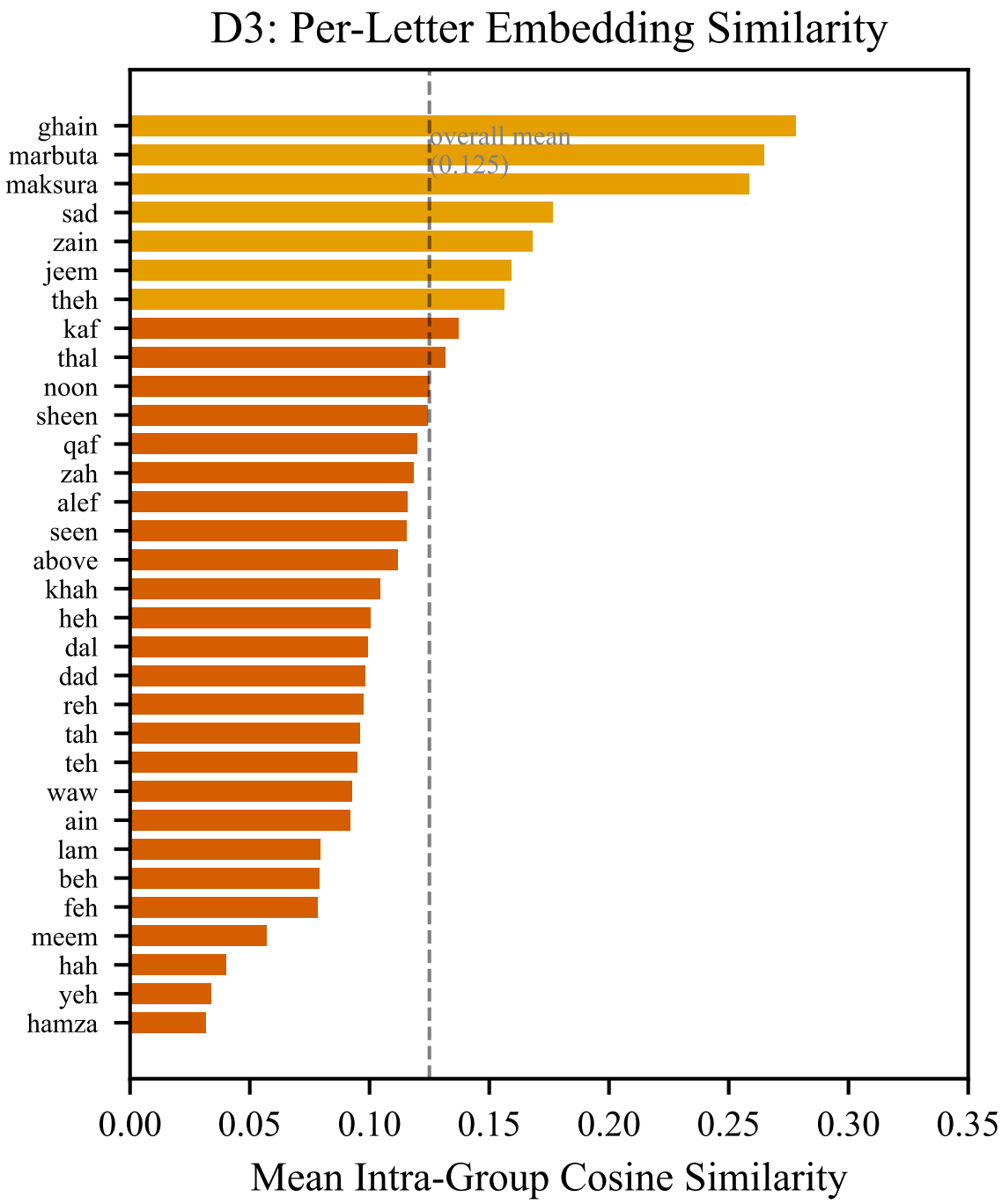

Figure 1: Intra-group cosine similarity for each of the 33 Arabic base letters in d3 embeddings. The dashed line marks the overall mean (0.125). Most letters have similarity well below 0.2, indicating the model treats diacritical variants as nearly independent. Letters with fewer variants (ghain, marbuta) show higher similarity.

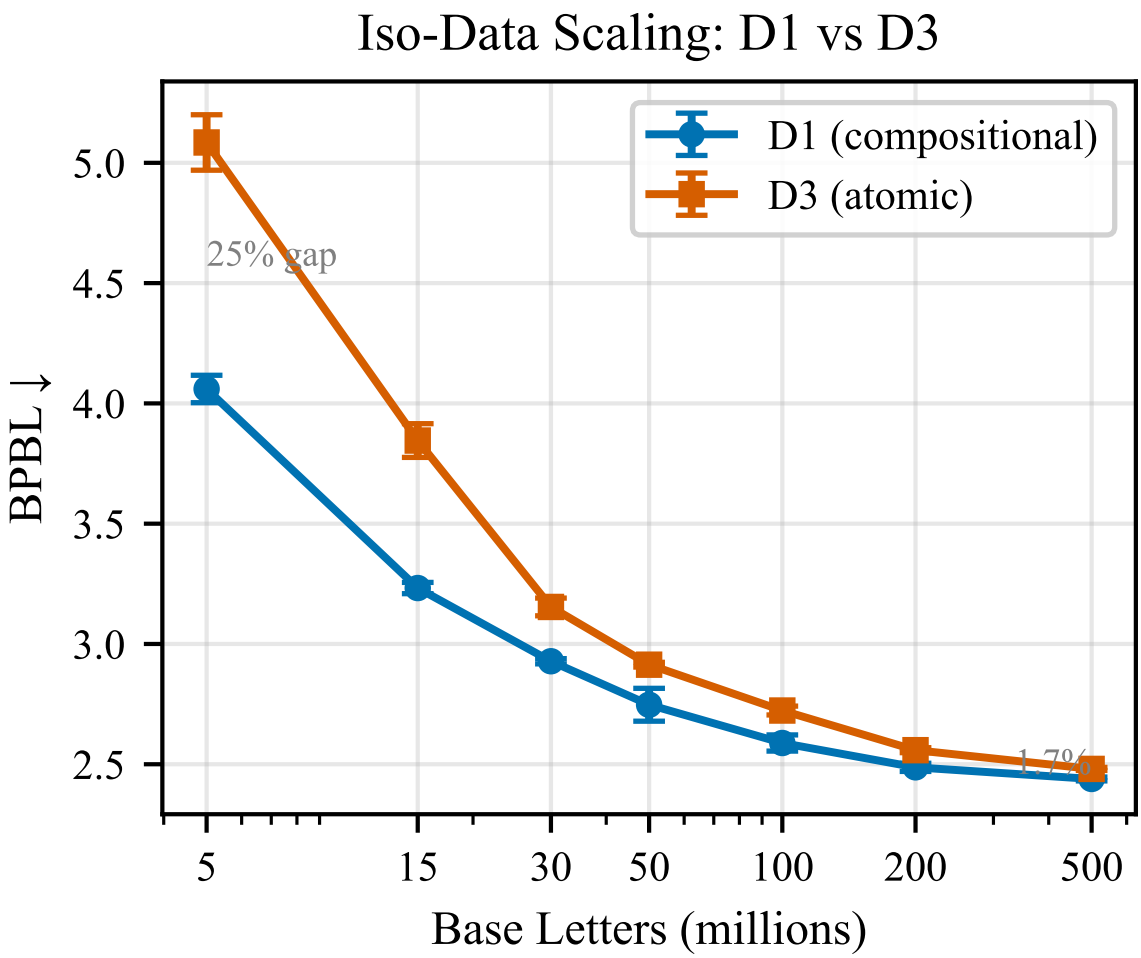

Figure 2: Iso-data scaling curves. Both d1 and d3 improve with more data, but the gap narrows monotonically from 25% to under 2%. Error bars show ±1 standard deviation across 3 seeds.

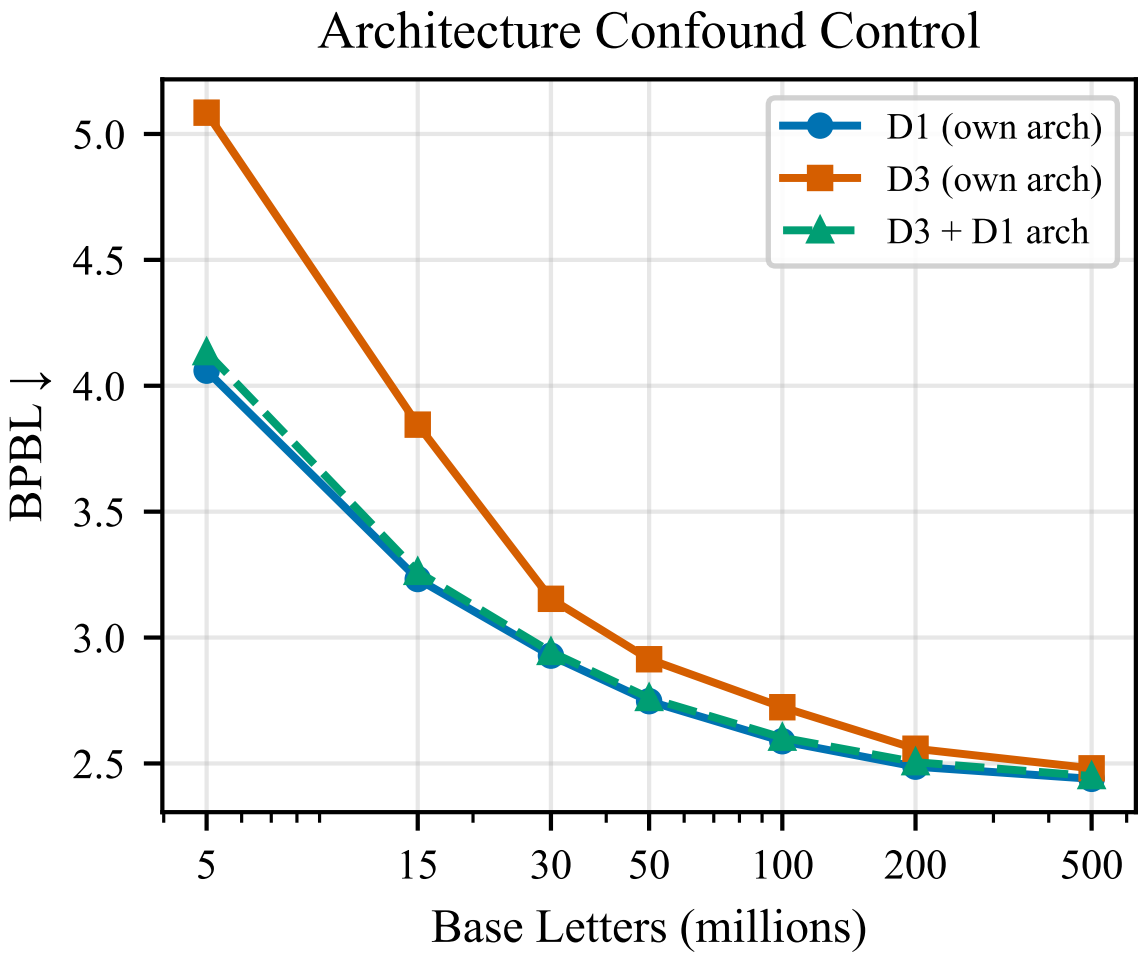

Figure 3: Architecture confound control. When d3 uses d1's deeper architecture (green triangles), it performs comparably to d1 at all scales, confirming the encoding effect diminishes when architecture is controlled.

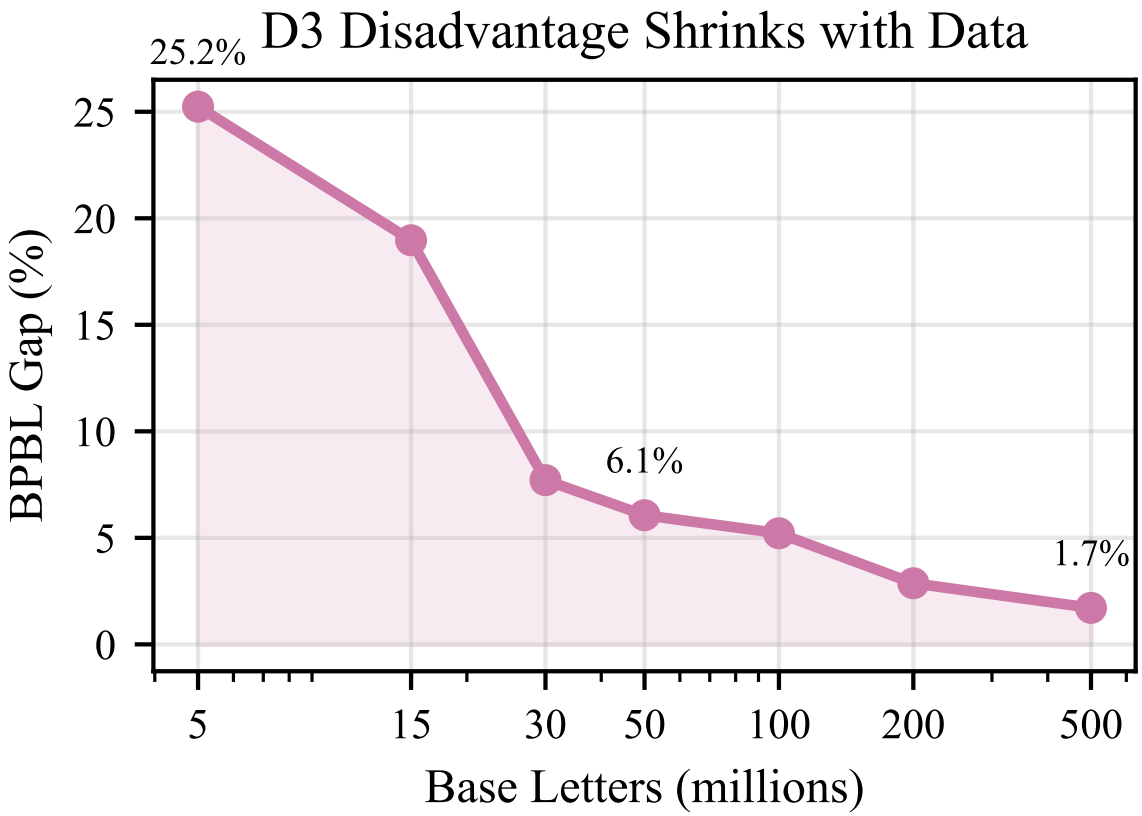

Figure 4: The d3 disadvantage (measured as percentage gap in bpbl) shrinks monotonically from 25% at 5M base letters to under 2% at 500M. The compositionality–atomicity tradeoff is data-scale dependent.